02393 Programming in C++

Module 12: Trees

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Trees
- Trees in C++
- Using trees to represent expressions
- Lab

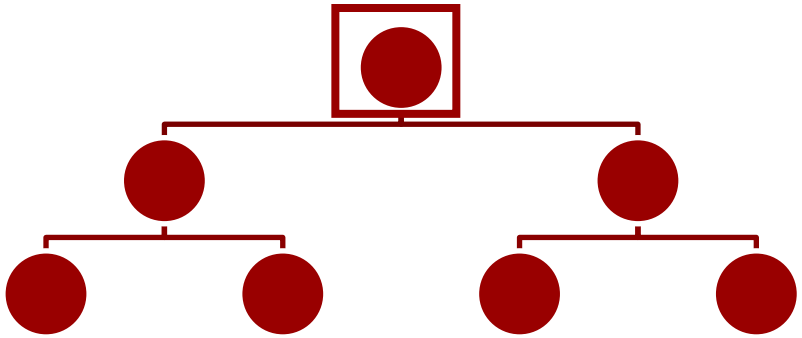
A recap from the previous lectures

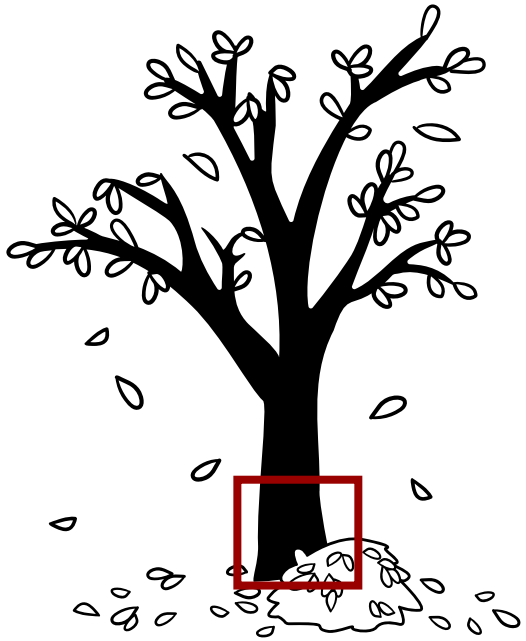
- The structure of a C++ program
  - #include and #define directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
  - cin, cout
- Variables, values and types
  - string, int, double, float, bool, arrays (statically and dynamically allocated), pointers, enum, struct, vector, ifstream, ofstream, class, this
- Expressions
  - some numeric and boolean operators and math functions, conditional expressions
- Statements
  - if, while, for, switch
- Recursive programming
  - Recursive functions, linked lists

Recursive programming

- Recursive programming is useful when dealing with
  - Recursively-defined problems (e.g. computing the factorial of a number)
  - Recursive data structures
- Examples of recursive data structures:
  - Linked lists (last lecture): singly-linked and doubly-linked
  - Trees (today!)
  - Sets, multi-sets
- Always keep in mind the difference between specification vs. implementation!
  - Abstract Data Type: a type being specified
    - E.g. a class MyVector
  - Concrete Data Structure: how we implement a data type
    - E.g. how the MyVector class internally stores its data, by using arrays, or linked lists, etc.

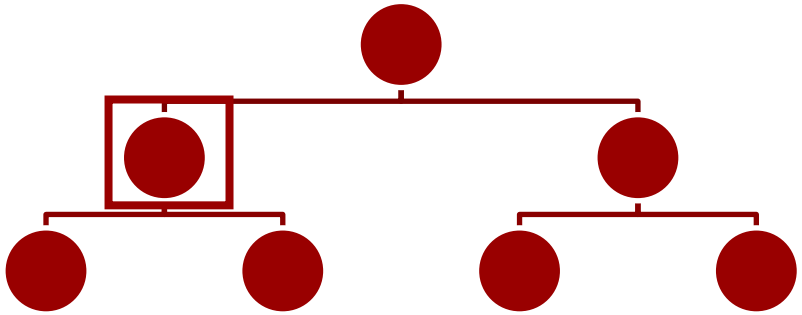
Trees

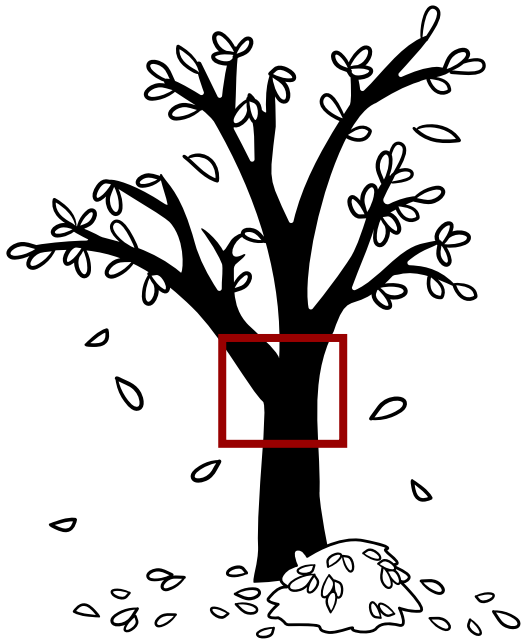




- Data structures useful to represent hierarchical dependencies
  - Root node: topmost node of a tree
    - Has no parent node
    - Has at least one child node

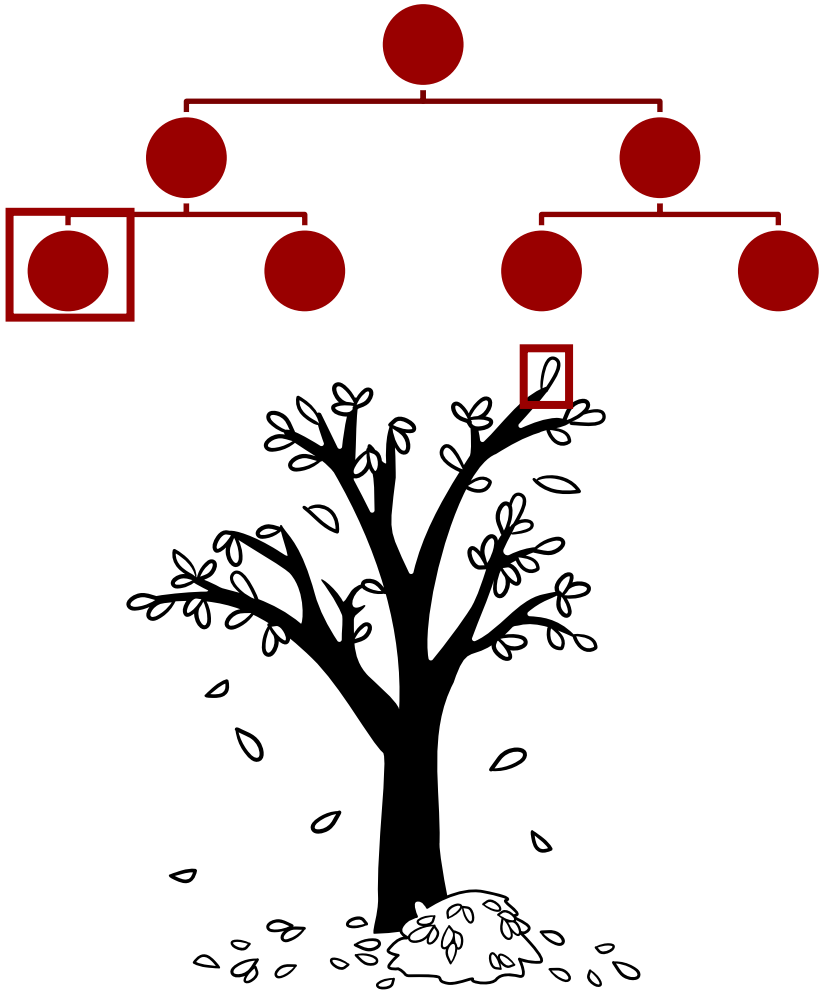
Trees





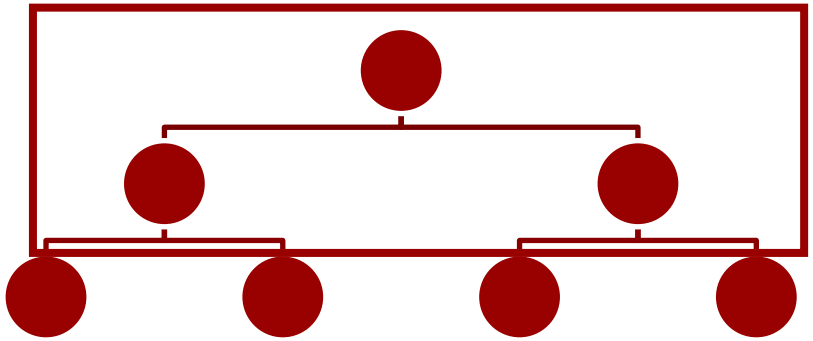
- Data structures useful to represent hierarchical dependencies
  - Root node: topmost node of a tree
    - Has no parent node
    - Has at least one child node
  - Intermediate node:
    - Has one parent node
    - Has one or more child nodes

Trees



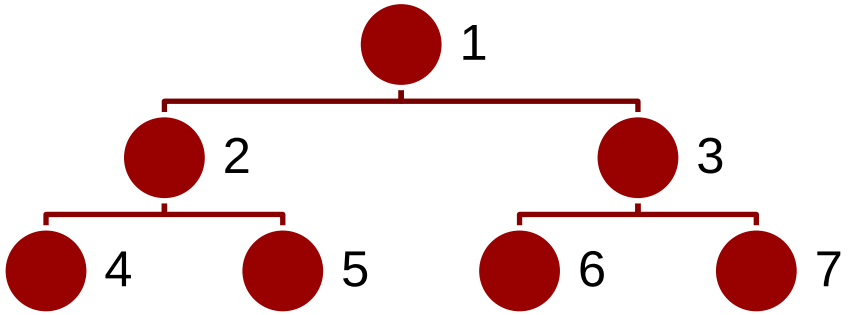
- Data structures useful to represent hierarchical dependencies
  - Root node: topmost node of a tree
    - Has no parent node
    - Has at least one child node
  - Intermediate node:
    - Has one parent node
    - Has one or more child nodes
  - Leaf node:
    - Has one parent node
    - Has no child nodes

Trees



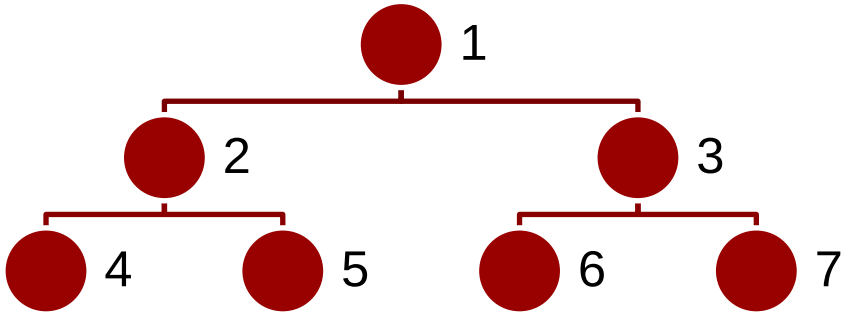
- Internal nodes:
  - Root node + intermediate nodes

Trees



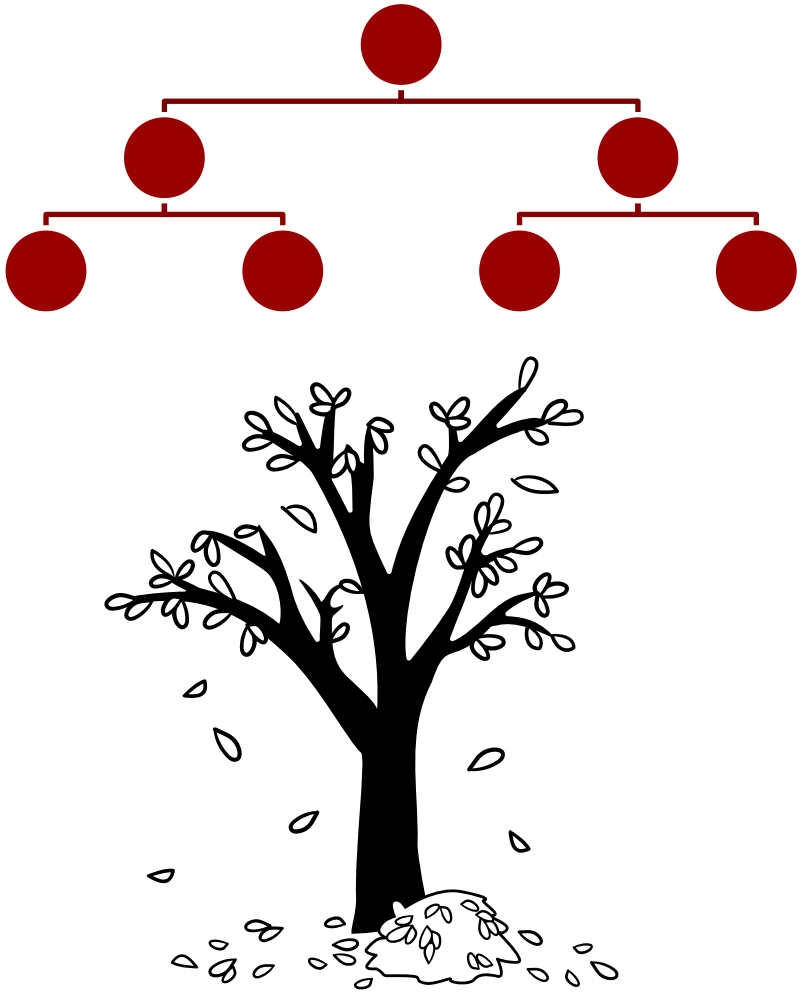
- Internal nodes:
  - Root node + intermediate nodes
- Size of a tree:
  - Number of nodes in a tree

Trees



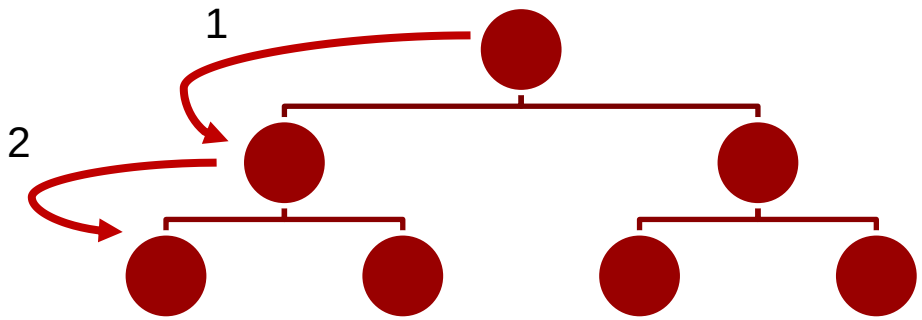
- Internal nodes:
  - Root node + intermediate nodes
- Size of a tree:
  - Number of nodes in a tree
  - In this example, size is 7

Trees



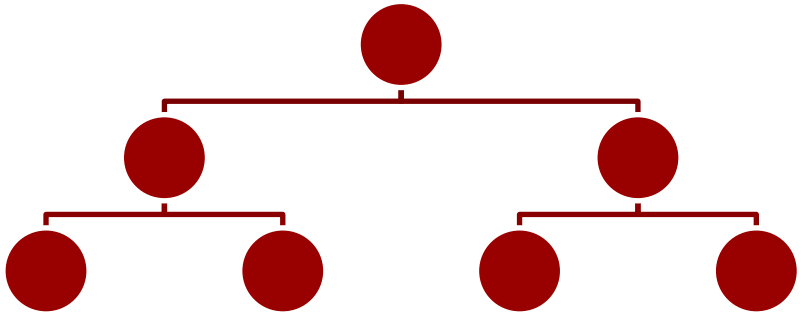
- Internal nodes:
  - Root node + intermediate nodes
- Size of a tree:
  - Number of nodes in a tree
  - In this example, size is 7
- Height of a tree:
  - Length (number of edges) of the longest path from the root to a leaf

Trees



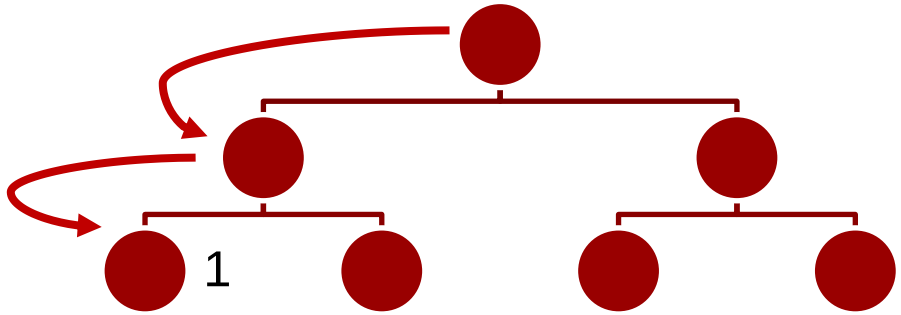
- Internal nodes:
  - Root node + intermediate nodes
- Size of a tree:
  - Number of nodes in a tree
  - In this example, size is 7
- Height of a tree:
  - Length (number of edges) of the longest path from the root to a leaf
  - In this example, size is 2

Trees



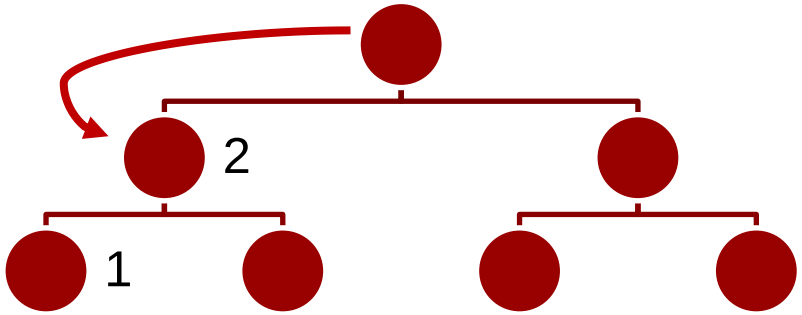
- Internal nodes:
  - Root node + intermediate nodes
- Size of a tree:
  - Number of nodes in a tree
  - In this example, size is 7
- Height of a tree:
  - Length (number of edges) of the longest path from the root to a leaf
  - In this example, size is 2
- Binary tree:
  - Tree whose nodes (except leaf nodes) have exactly 2 child nodes

Trees



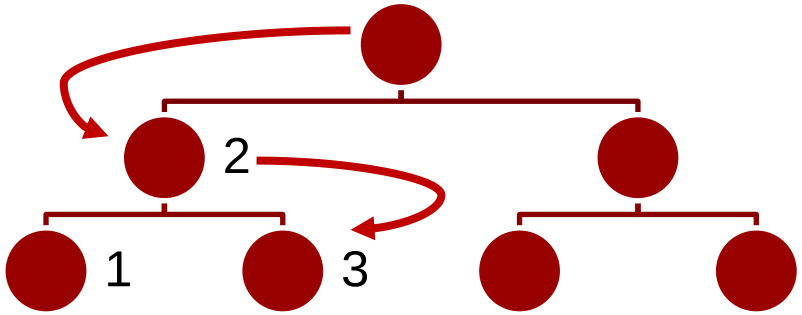
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



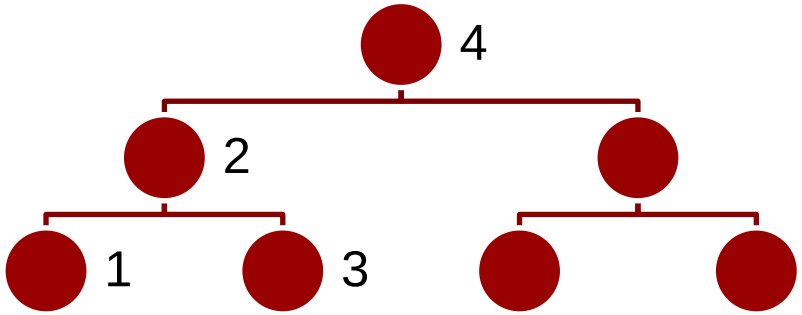
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



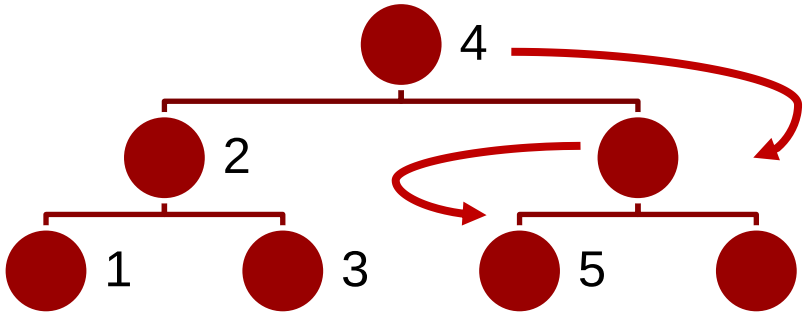
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



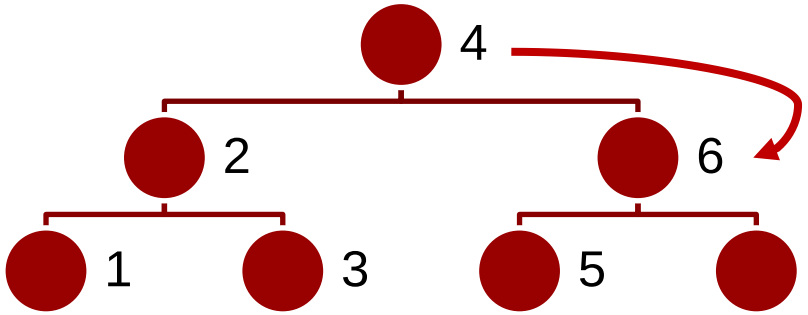
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



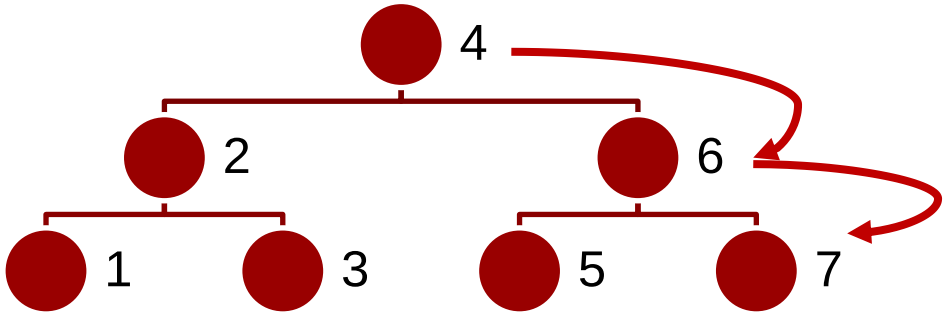
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



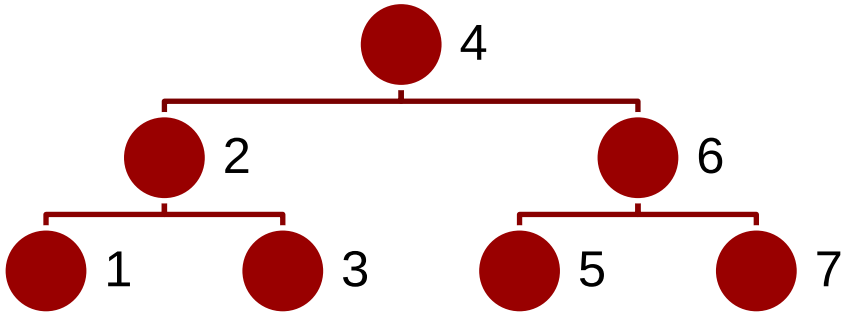
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



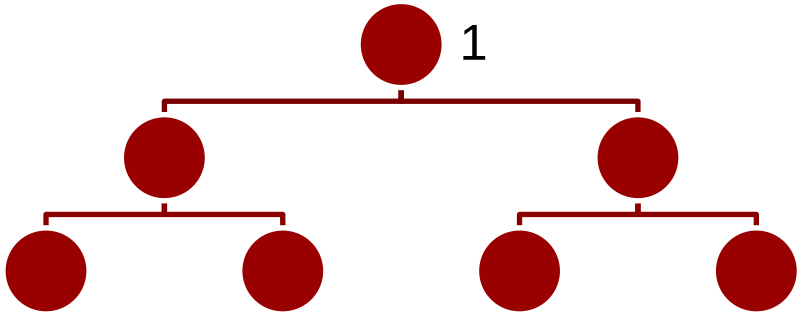
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



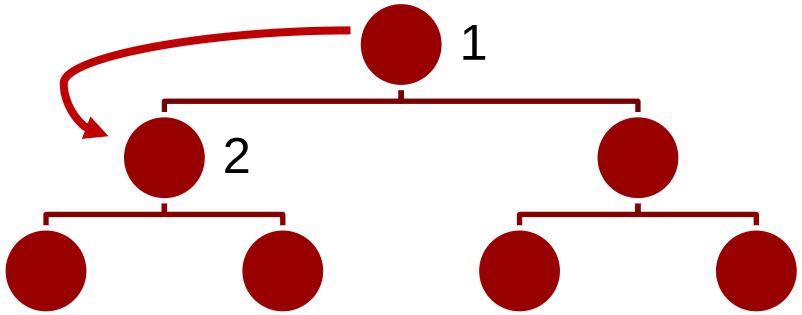
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent

Trees



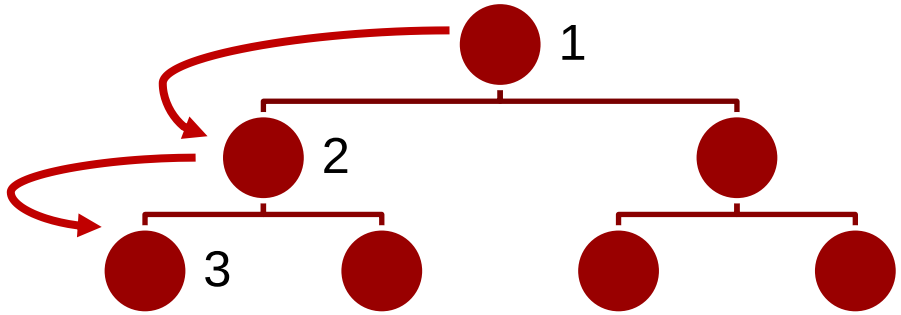
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



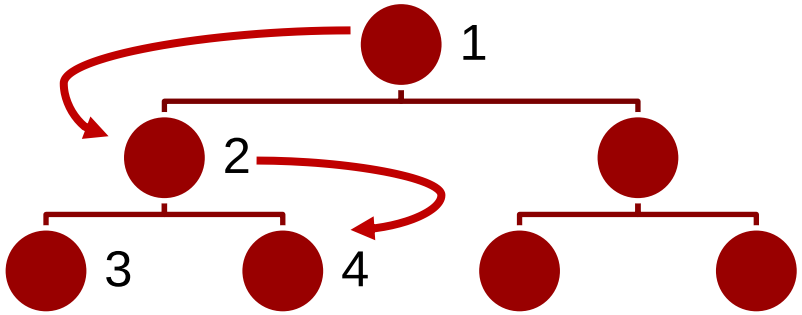
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



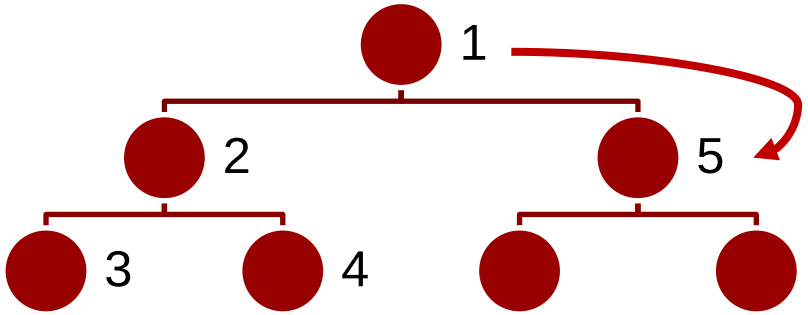
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



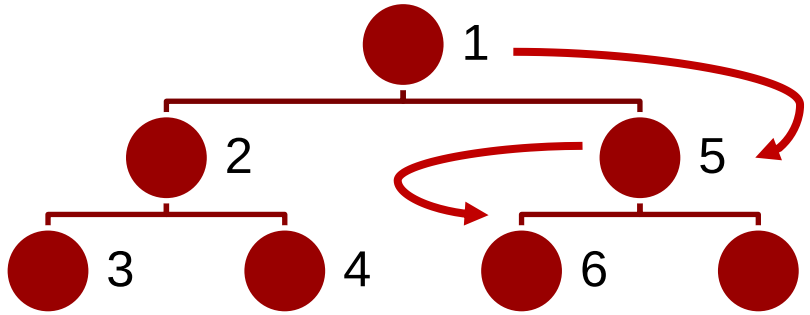
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



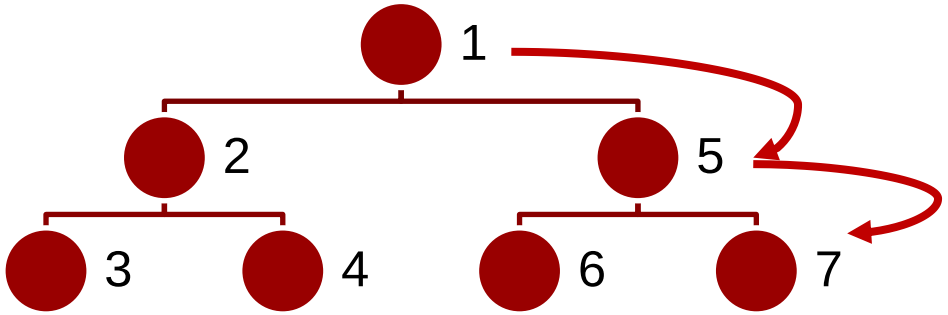
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



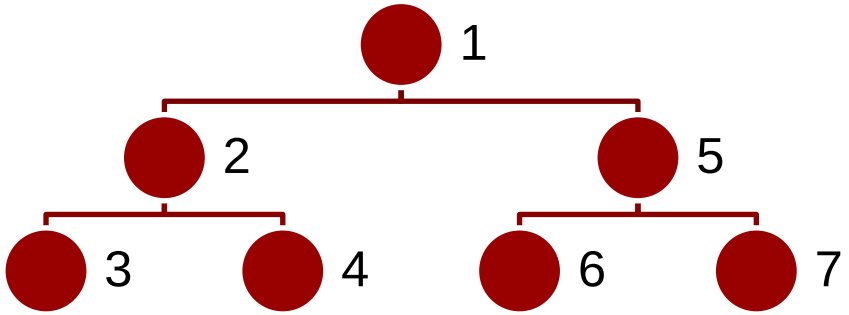
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



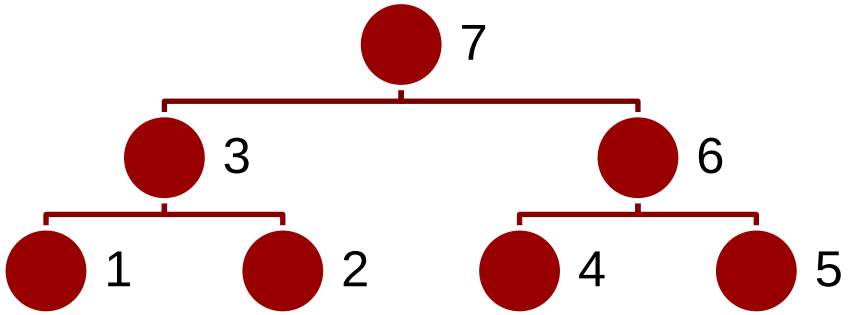
- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent

Trees



- In-order traversal:
  - Traverse the left descendent
  - Show the current node contents
  - Traverse the right descendent
- Pre-order traversal:
  - Show the current node contents
  - Traverse the left descendent
  - Traverse the right descendent
- Post-order traversal:
  - Traverse the left descendent
  - Traverse the right descendent
  - Show the current node contents

Representing trees in C++

```
1  enum NodeType { Internal, Leaf };
2
3  struct Node {
4      NodeType type;
5      int value; // Only used if type == Leaf
6      Node *left; // Only used if type != Leaf
7      Node *right; // Only used if type != Leaf
8  }
9
10 class Node {
11 public:
12     //methods to handle the tree
13
14 private:
15     NodeType type;
16     int value; // Only used if type == Leaf
17     Node *left; // Only used if type != Leaf
18     Node *right; // Only used if type != Leaf
19 }
```

- Similar to a single-linked list:
  - Each node is represented with a struct or a class

Representing trees in C++

```
1  enum NodeType { Internal, Leaf };
2
3  struct Node {
4      NodeType type;
5      int value; // Only used if type == Leaf
6      Node *left; // Only used if type != Leaf
7      Node *right; // Only used if type != Leaf
8  }
9
10 class Node {
11 public:
12     //methods to handle the tree
13
14 private:
15     NodeType type;
16     int value; // Only used if type == Leaf
17     Node *left; // Only used if type != Leaf
18     Node *right; // Only used if type != Leaf
19 }
```

- Similar to a single-linked list:
  - Each node is represented with a struct or a class
  - Each node has two pointers to the child nodes

Representing trees in C++

```
1  enum NodeType { Internal, Leaf };
2
3  struct Node {
4      NodeType type;
5      int value; // Only used if type == Leaf
6      Node *left; // Only used if type != Leaf
7      Node *right; // Only used if type != Leaf
8  }
9
10 class Node {
11 public:
12     //methods to handle the tree
13
14 private:
15     NodeType type;
16     int value; // Only used if type == Leaf
17     Node *left; // Only used if type != Leaf
18     Node *right; // Only used if type != Leaf
19 }
```

- Similar to a single-linked list:
  - Each node is represented with a struct or a class
  - Each node has two pointers to the child nodes
  - (optional) Each node is identified as leaf or internal node
    - Alternatively, to identify leaf nodes, one needs to set both pointers to child nodes to nullptr

Computing the size of a tree in C++

```
1  enum NodeType { Internal, Leaf };
2
3  struct Node {
4      NodeType type;
5      int value; // Only used if type == Leaf
6      Node *left; // Only used if type != Leaf
7      Node *right; // Only used if type != Leaf
8  }
9
10 int size(Node *node) {
11     if (node->type == Leaf) {
12         return 1;
13     } else {
14         return 1 + size(node->left) + size(node->right);
15     }
16 }
```

- Current node is a leaf node (base case)
  - Size is 1

Computing the size of a tree in C++

```
1   enum NodeType { Internal, Leaf };
2
3   struct Node {
4       NodeType type;
5       int value; // Only used if type == Leaf
6       Node *left; // Only used if type != Leaf
7       Node *right; // Only used if type != Leaf
8   }
9
10  int size(Node *node) {
11      if (node->type == Leaf) {
12          return 1;
13      } else {
14          return 1 + size(node->left) + size(node->right);
15      }
16  }
```

- Current node is a leaf node (base case)
  - Size is 1
- Current node is an internal node:
  - Size is 1 + sizes of its child nodes

Computing the size of a tree in C++

```
1   enum NodeType { Internal, Leaf };
2
3   struct Node {
4       NodeType type;
5       int value; // Only used if type == Leaf
6       Node *left; // Only used if type != Leaf
7       Node *right; // Only used if type != Leaf
8   }
9
10  int size(Node *node) {
11      if (node->type == Leaf) {
12          return 1;
13      } else {
14          return 1 + size(node->left) + size(node->right);
15      }
16  }
```

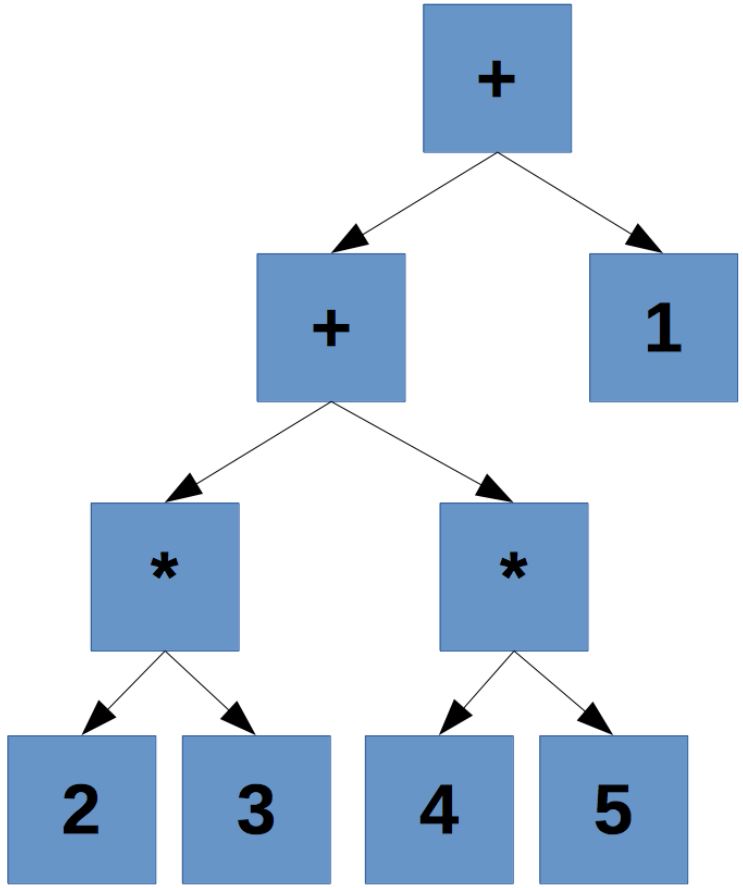
- Current node is a leaf node (base case)
  - Size is 1
- Current node is an internal node:
  - Size is 1 + sizes of its child nodes
- Non-recursive implementations are possible, but more complicated!

Using trees to represent expressions

- How can we represent the following expression?

  (2 * 3) + (4 * 5)) + 1

Using trees to represent expressions

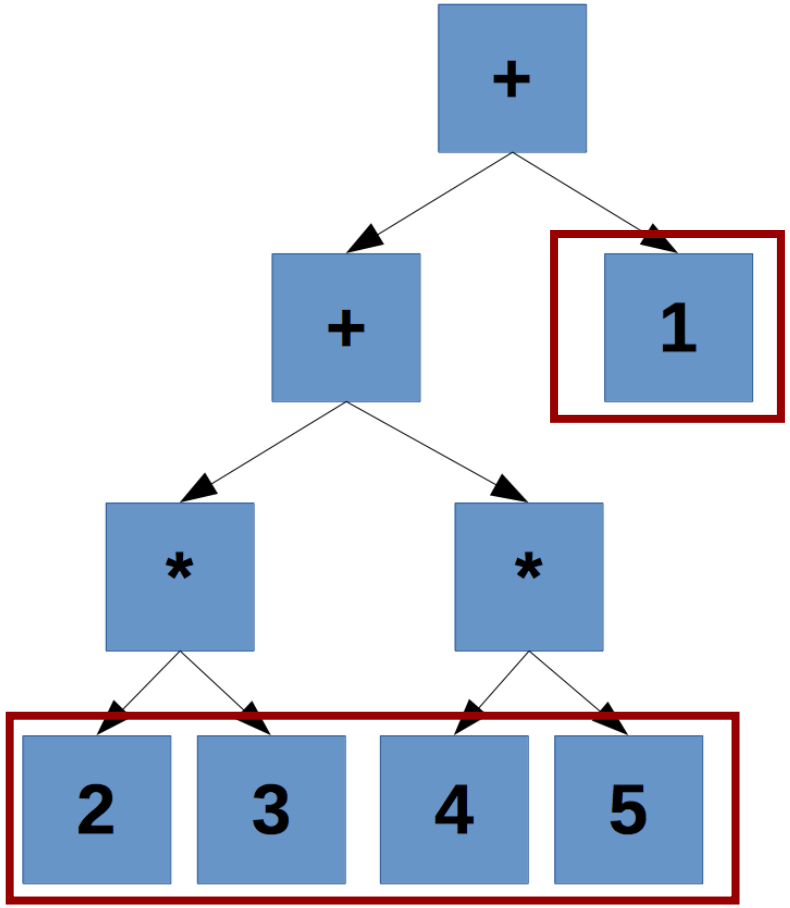


- How can we represent the following expression?

  (2 * 3) + (4 * 5)) + 1

- We can use a binary tree!

Using trees to represent expressions

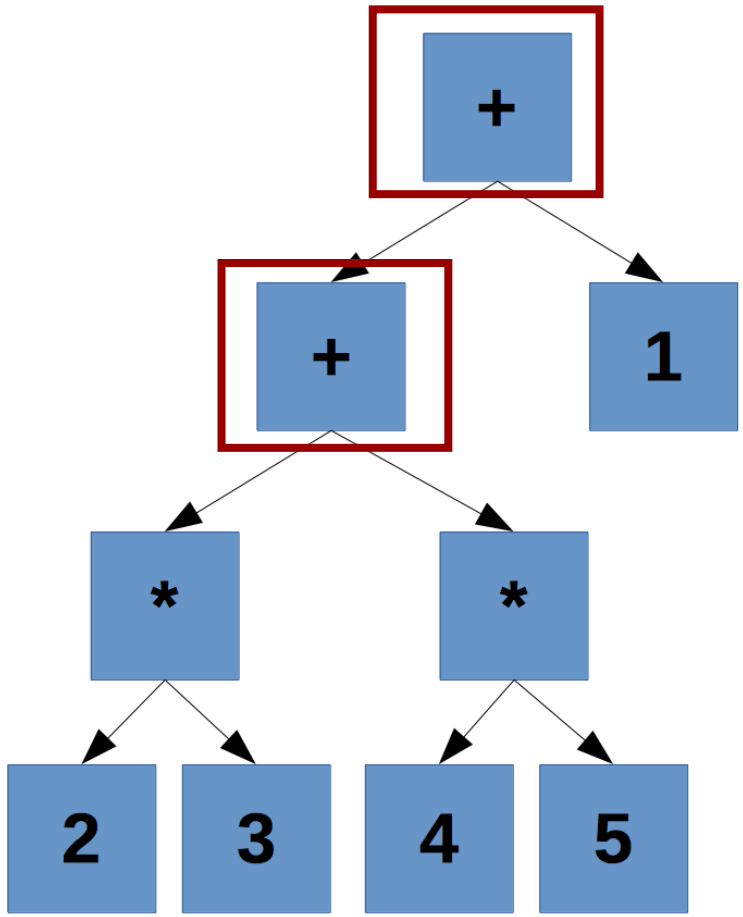


- How can we represent the following expression?

  (2 * 3) + (4 * 5)) + 1

- We can use a binary tree!
- We need to represent:
  – Constants

Using trees to represent expressions

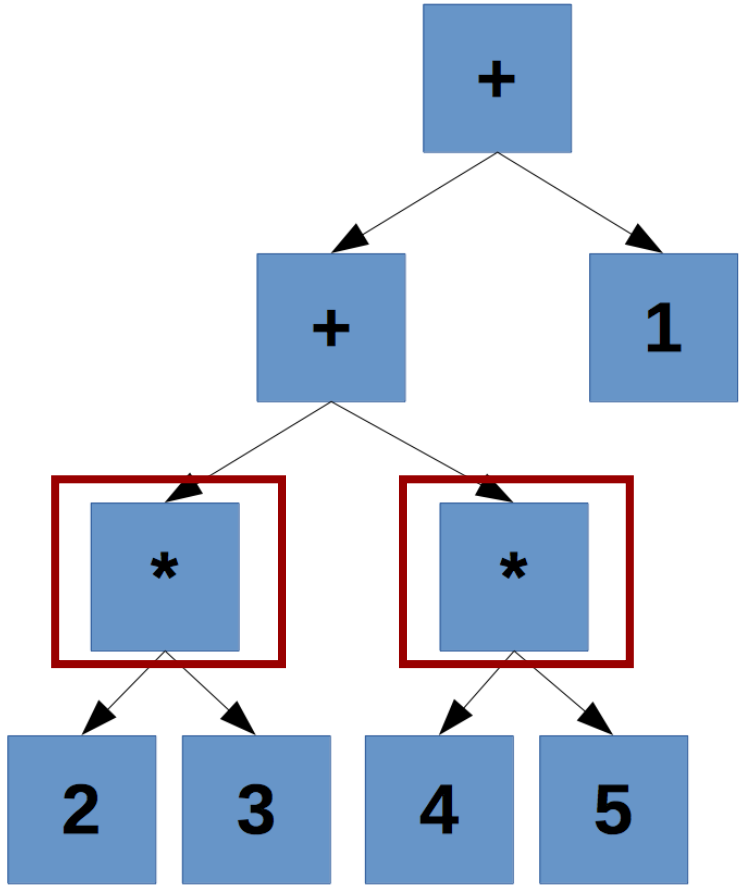


- How can we represent the following expression?

  (2 * 3) + (4 * 5)) + 1

- We can use a binary tree!
- We need to represent:
  - Constants
  - The sum of two arithmetic expressions

Using trees to represent expressions

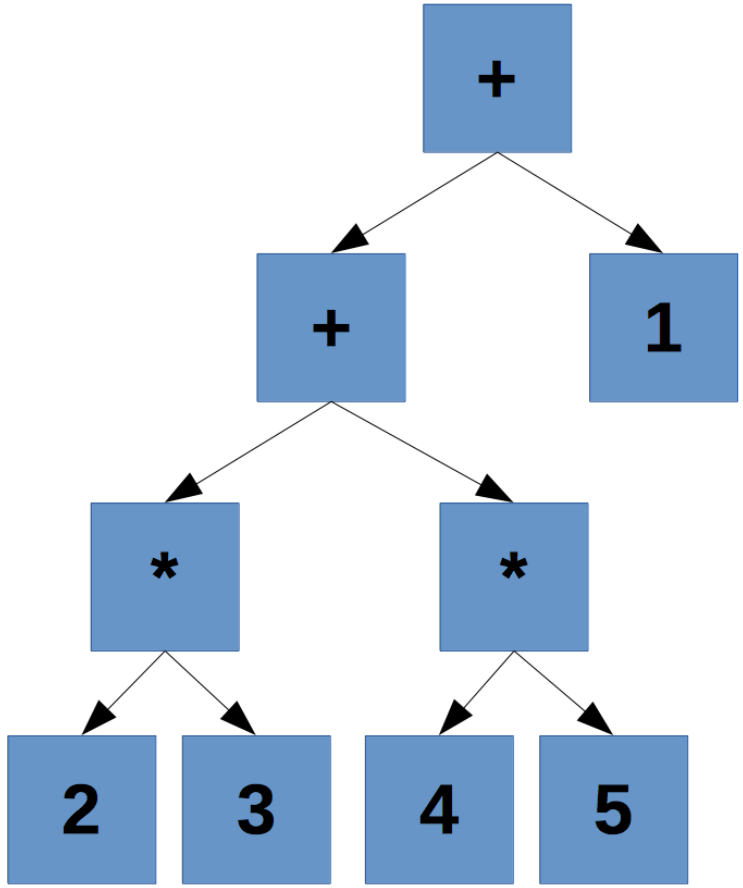


- How can we represent the following expression?

  (2 * 3) + (4 * 5)) + 1

- We can use a binary tree!
- We need to represent:
  - Constants
  - The sum of two arithmetic expressions
  - The product of two arithmetic expressions

Using trees to represent expressions



- During live coding session, we will see how to:
  - Compute size
  - Compute height
  - Compute number of leaves
  - Traverse the tree in different ways:
    - In-order: print algebraic notation
    - Pre-order
    - Post-order: print reverse polish notation

Lab time

- Today
  - Make sure C++ works on your computer, request help if it doesn’t
  - Begin working on Assignment 11
  - Ask questions if something is unclear (including previous assignments)
  - Make sure you can run the local autograder for May 2021 and September 2021 exams


DTU